

Convolutional Neural Networks (CNNs)

by Dr. Rishikesh Yadav

June 03, 2026

Assistant Professor, School of Mathematical and Statistical Sciences, IIT Mandi, India

Table of Contents

- ✓ 1. Neural Networks: Multi-Layer Perceptron (MLP) → ^{NNs} → ^{FNN}
- ✓ 1.1 Recap MLPs (Fully Connected NNs + Feedforward) →
- 1.2 Motivation: why CNNs for Images?
- 2. Convolutional Neural Networks (CNNs)
- 2.1 From ^{NN} MLP to CNN
- 2.2 Core Components of CNNs
 - Convolution ✓
 - Pooling ✓
 - Fully Connected Layers ✓
- 2.3 Full CNN Pipeline
- 2.4 Training CNNs
- 2.5 CNNs Lab ✓

Neural Networks: Multi-Layer Perceptron (MLP)

→ FNN

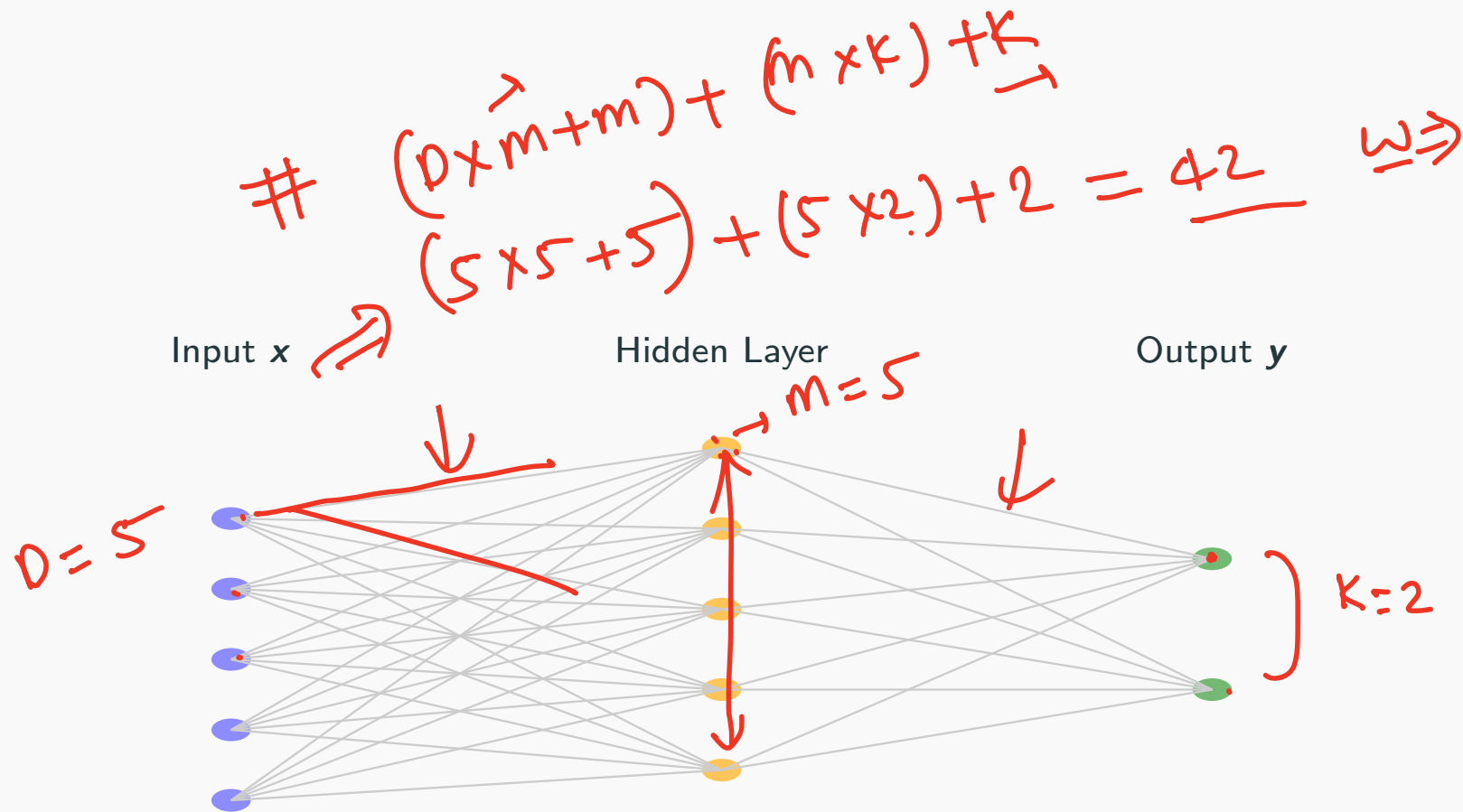
Agenda

1. Neural Networks: Multi-Layer Perceptron (MLP)
 - 1.1 Recap MLPs (Fully Connected NNs + Feedforward)
 - 1.2 Motivation: why CNNs for Images?

Quick Recap: a Basic Neural Network (MLP)

- A (fully-connected) layer maps an input vector $\underline{x} \in \mathbb{R}^D$ to $\underline{y} \in \mathbb{R}^K$:
 $x_1 \dots x_D$

$$\mathbf{y} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$$

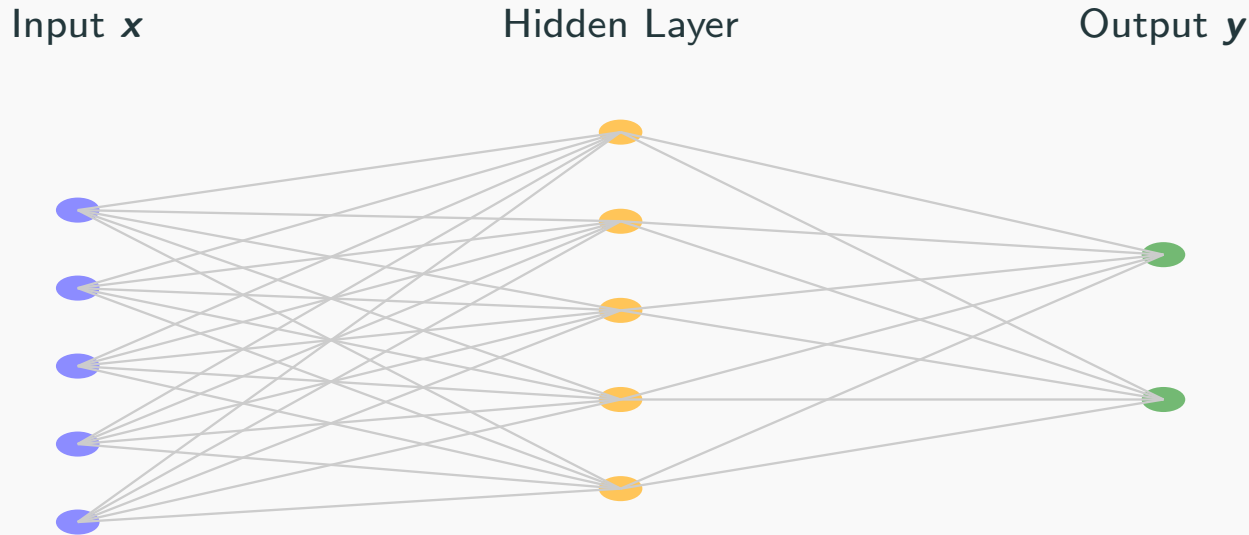


Quick Recap: a Basic Neural Network (MLP)

- A (fully-connected) layer maps an input vector $\mathbf{x} \in \mathbb{R}^D$ to $\mathbf{y} \in \mathbb{R}^K$:

$$\mathbf{y} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$$

- **Parameters:** $(d \times M + M \times K)$ weights + $(M + K)$ biases. Where M is the number of hidden units (nodes).

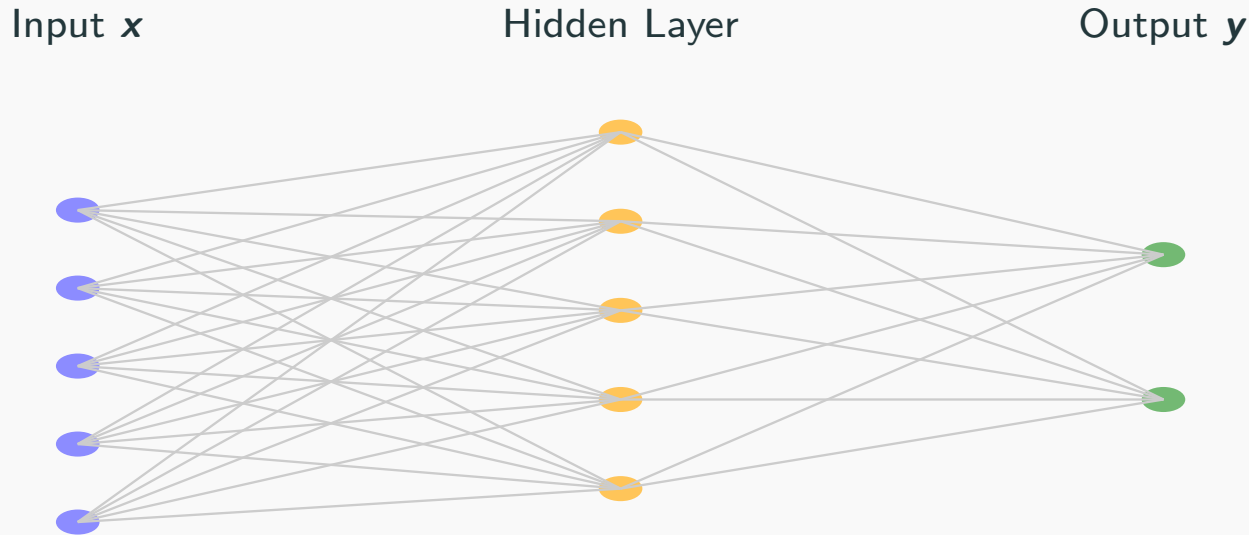


Quick Recap: a Basic Neural Network (MLP)

- A (fully-connected) layer maps an input vector $\mathbf{x} \in \mathbb{R}^D$ to $\mathbf{y} \in \mathbb{R}^K$:

$$\mathbf{y} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$$

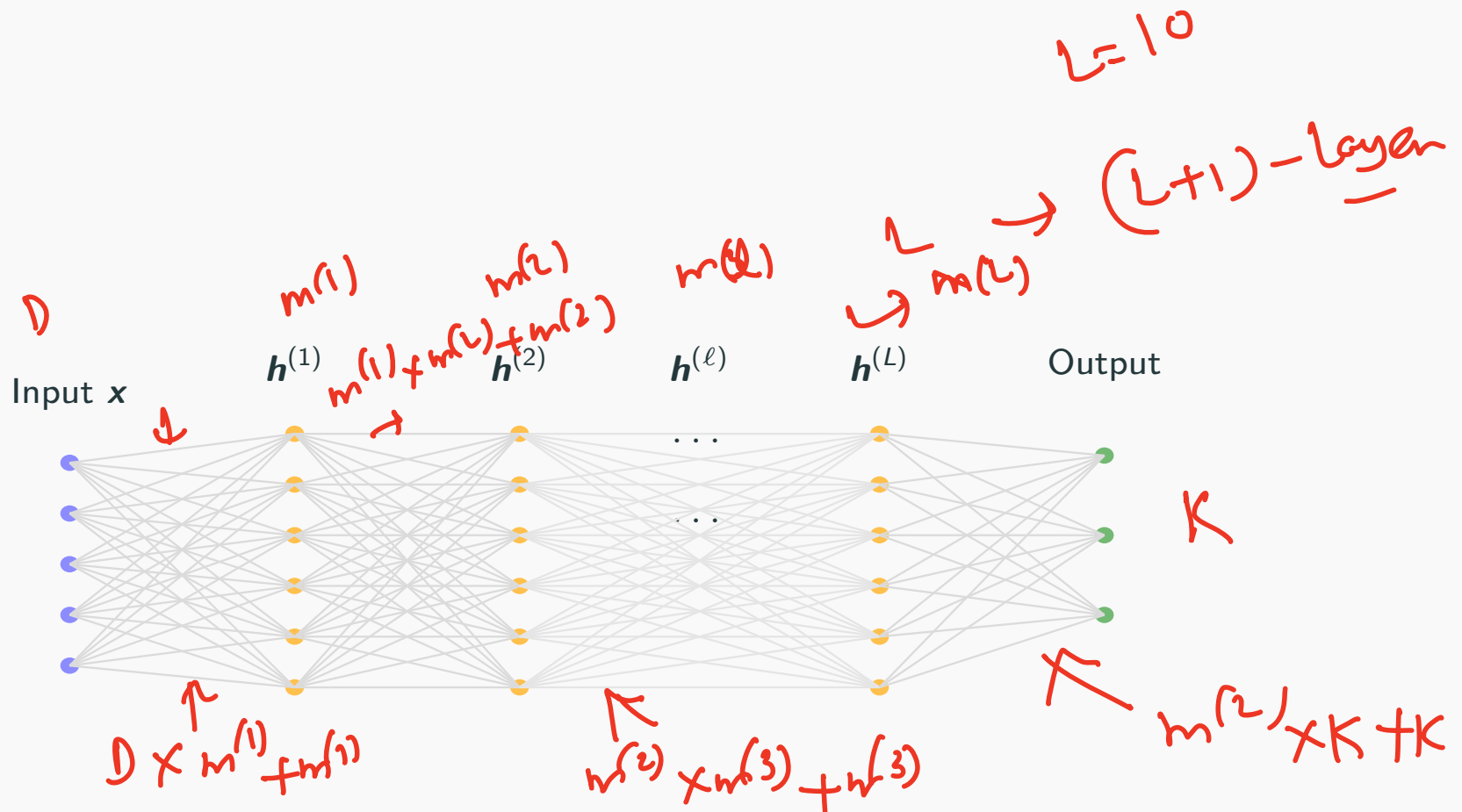
- **Parameters:** $(d \times M + M \times K)$ weights + $(M + K)$ biases. Where M is the number of hidden units (nodes).
- MLPs treat inputs as 1D vectors (no explicit spatial structure).



Deep Neural Networks: L Hidden Layers

- A deep MLP is a composition of affine maps (linear transformation + bias) + nonlinearities.

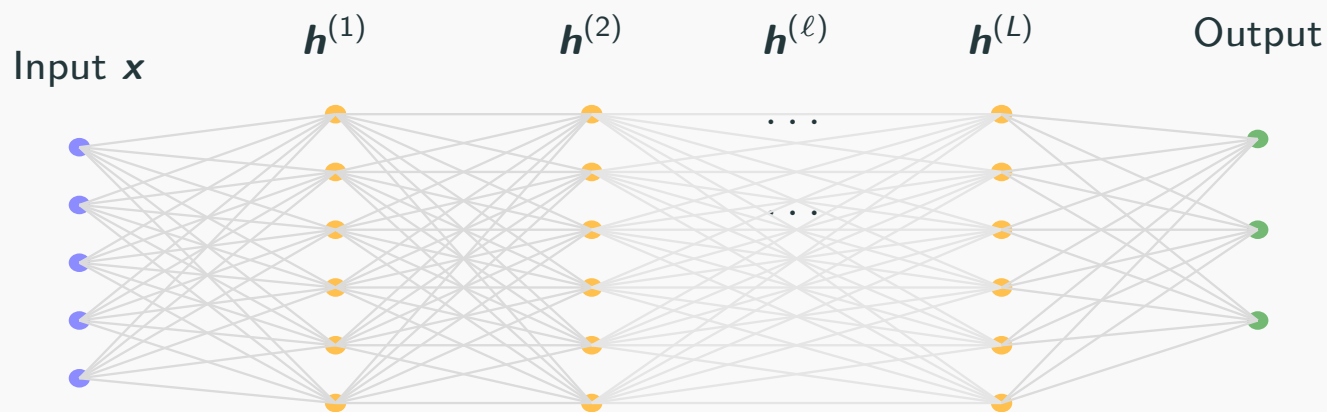
$\Rightarrow$



Deep Neural Networks: L Hidden Layers

- A deep MLP is a composition of affine maps (linear transformation + bias) + nonlinearities.
- Let $\mathbf{h}^{(0)} = \mathbf{x}$ and for $\ell = 1, \dots, L$:

$$\mathbf{z}^{(\ell)} = \mathbf{W}^{(\ell)} \mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \quad \mathbf{h}^{(\ell)} = \sigma(\mathbf{z}^{(\ell)})$$

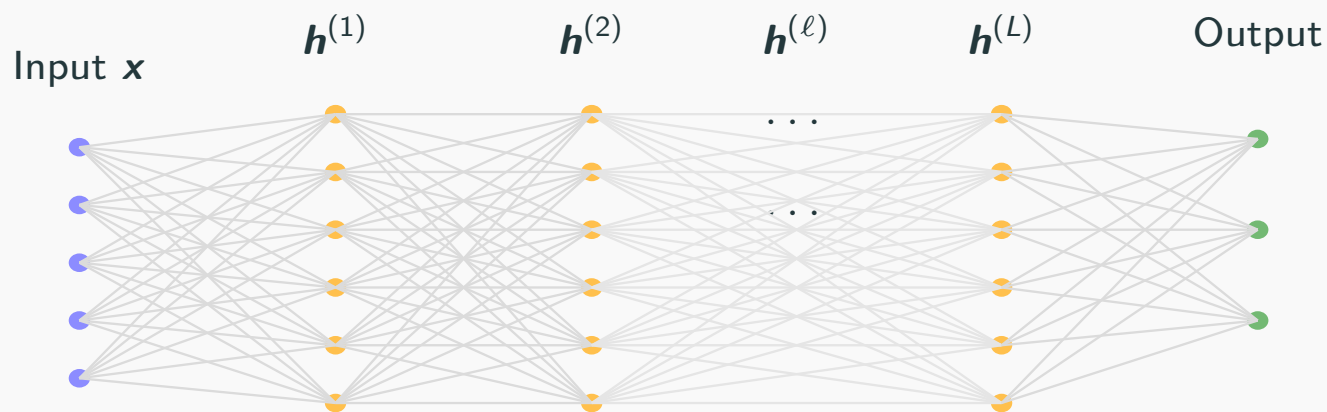


Deep Neural Networks: L Hidden Layers

- A deep MLP is a composition of affine maps (linear transformation + bias) + nonlinearities.
- Let $\mathbf{h}^{(0)} = \mathbf{x}$ and for $\ell = 1, \dots, L$:

$$\mathbf{z}^{(\ell)} = \mathbf{W}^{(\ell)} \mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \quad \mathbf{h}^{(\ell)} = \sigma(\mathbf{z}^{(\ell)})$$

- Output layer (example for classification):
 $\mathbf{z}^{(L+1)} = \sigma(\mathbf{W}^{(L+1)} \mathbf{h}^{(L)} + \mathbf{b}^{(L+1)})$; here σ must be the softmax function.



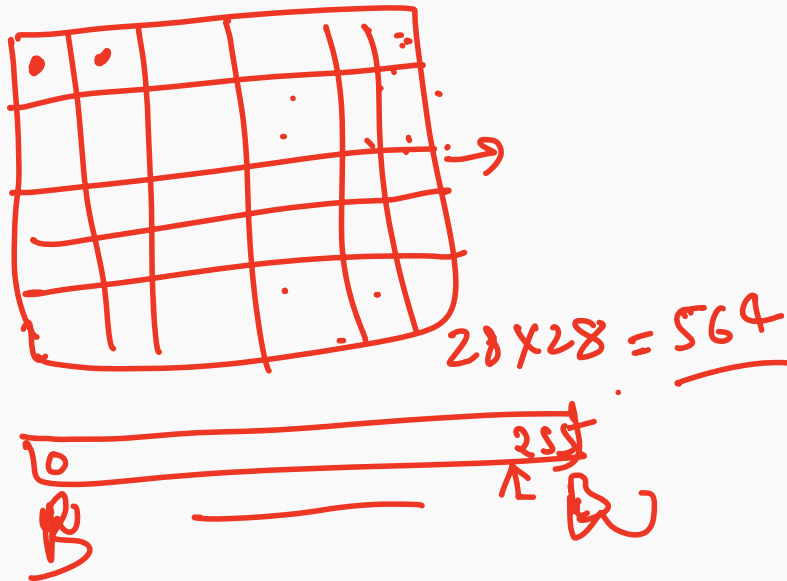
Agenda

1. Neural Networks: Multi-Layer Perceptron (MLP)
 - 1.1 Recap MLPs (Fully Connected NNs + Feedforward)
 - 1.2 Motivation: why CNNs for Images?

Grayscale vs. RGB Images (How Computers Represent Them)

Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black → white).



Grayscale vs. RGB Images (How Computers Represent Them)

Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.

Grayscale vs. RGB Images (How Computers Represent Them)

Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.
- Typical 8-bit encoding: values in $[0, 255]$

Grayscale vs. RGB Images (How Computers Represent Them)

Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.
- Typical 8-bit encoding: values in $[0, 255]$
 - 0 = black

Grayscale vs. RGB Images (How Computers Represent Them)

Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.
- Typical 8-bit encoding: values in $[0, 255]$
 - 0 = black
 - 255 = white

Grayscale vs. RGB Images (How Computers Represent Them)

Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.
- Typical 8-bit encoding: values in $[0, 255]$
 - 0 = black ✓
 - 255 = white ✓
 - middle values = shades of gray

Grayscale vs. RGB Images (How Computers Represent Them)

Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.
- Typical 8-bit encoding: values in $[0, 255]$
 - 0 = black
 - 255 = white
 - middle values = shades of gray
- Example: 128 $\rightarrow$ mid-gray

Grayscale vs. RGB Images (How Computers Represent Them)

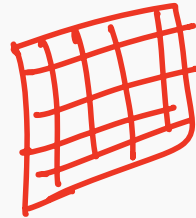
Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.
- Typical 8-bit encoding: values in $[0, 255]$
 - 0 = black
 - 255 = white
 - middle values = shades of gray
- Example: 128 $\rightarrow$ mid-gray

Grayscale vs. RGB Images (How Computers Represent Them)

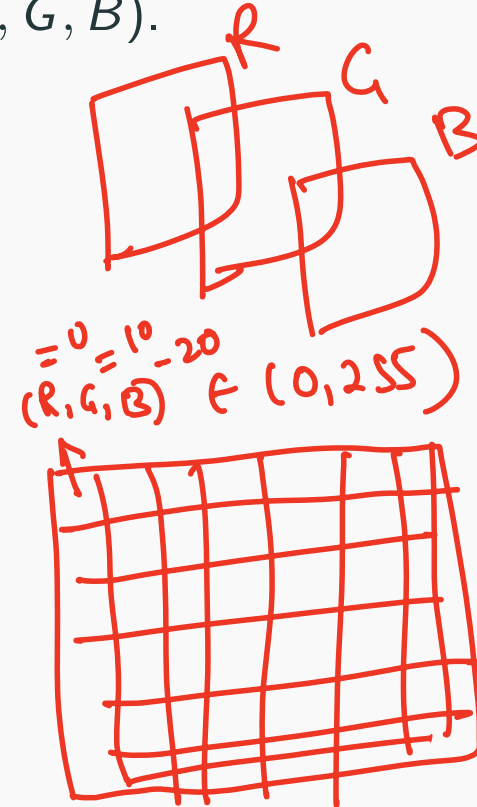
Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.
- Typical 8-bit encoding: values in $[0, 255]$
 - 0 = black ✓
 - 255 = white
 - middle values = shades of gray
- Example: 128 $\rightarrow$ mid-gray



RGB (3 channels)

- Each pixel stores a **3-tuple** (R, G, B) .



Grayscale vs. RGB Images (How Computers Represent Them)

Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.
- Typical 8-bit encoding: values in $[0, 255]$
 - 0 = black
 - 255 = white
 - middle values = shades of gray
- Example: 128 $\rightarrow$ mid-gray

RGB (3 channels)

- Each pixel stores a **3-tuple** (R, G, B) .
- Each channel has values in $[0, 255]$ (8-bit each).

Grayscale vs. RGB Images (How Computers Represent Them)

Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.
- Typical 8-bit encoding: values in $[0, 255]$
 - 0 = black
 - 255 = white
 - middle values = shades of gray
- Example: 128 $\rightarrow$ mid-gray

RGB (3 channels)

- Each pixel stores a **3-tuple** (R, G, B) .
- Each channel has values in $[0, 255]$ (8-bit each).
- Stored as tensor $X \in \mathbb{R}^{H \times W \times 3}$.

Grayscale vs. RGB Images (How Computers Represent Them)

Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.
- Typical 8-bit encoding: values in $[0, 255]$
 - 0 = black
 - 255 = white
 - middle values = shades of gray
- Example: 128 $\rightarrow$ mid-gray

RGB (3 channels)

- Each pixel stores a **3-tuple** (R, G, B) .
- Each channel has values in $[0, 255]$ (8-bit each).
- Stored as tensor $X \in \mathbb{R}^{H \times W \times 3}$.
- Examples:

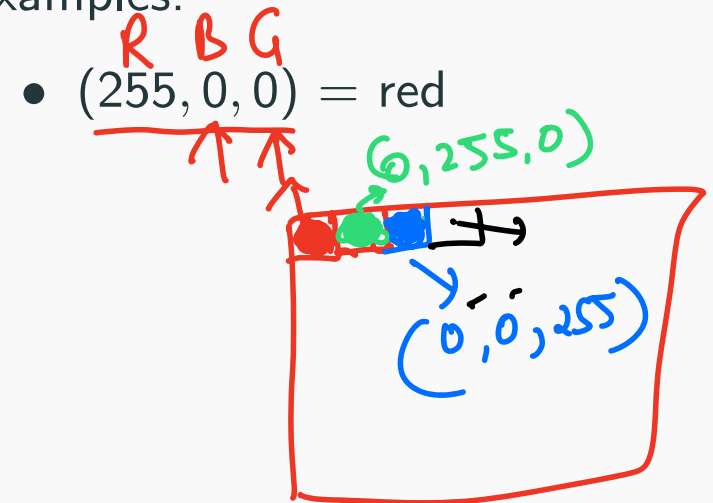
Grayscale vs. RGB Images (How Computers Represent Them)

Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.
- Typical 8-bit encoding: values in $[0, 255]$
 - 0 = black
 - 255 = white
 - middle values = shades of gray
- Example: 128 $\rightarrow$ mid-gray

RGB (3 channels)

- Each pixel stores a **3-tuple** (R, G, B) .
- Each channel has values in $[0, 255]$ (8-bit each).
- Stored as tensor $X \in \mathbb{R}^{H \times W \times 3}$.
- Examples:



Grayscale vs. RGB Images (How Computers Represent Them)

Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.
- Typical 8-bit encoding: values in $[0, 255]$
 - 0 = black
 - 255 = white
 - middle values = shades of gray
- Example: 128 $\rightarrow$ mid-gray

RGB (3 channels)

- Each pixel stores a **3-tuple** (R, G, B) .
- Each channel has values in $[0, 255]$ (8-bit each).
- Stored as tensor $X \in \mathbb{R}^{H \times W \times 3}$.
- Examples:
 - $(255, 0, 0)$ = red
 - $(0, \underline{255}, 0)$ = green

Grayscale vs. RGB Images (How Computers Represent Them)

Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.
- Typical 8-bit encoding: values in $[0, 255]$
 - 0 = black
 - 255 = white
 - middle values = shades of gray
- Example: 128 $\rightarrow$ mid-gray

RGB (3 channels)

- Each pixel stores a **3-tuple** (R, G, B) .
- Each channel has values in $[0, 255]$ (8-bit each).
- Stored as tensor $X \in \mathbb{R}^{H \times W \times 3}$.
- Examples:
 - $(255, 0, 0)$ = red
 - $(0, 255, 0)$ = green
 - $(0, 0, 255)$ = blue

Grayscale vs. RGB Images (How Computers Represent Them)

Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.
- Typical 8-bit encoding: values in $[0, 255]$
 - 0 = black
 - 255 = white
 - middle values = shades of gray
- Example: 128 $\rightarrow$ mid-gray

RGB (3 channels)

- Each pixel stores a **3-tuple** (R, G, B) .
- Each channel has values in $[0, 255]$ (8-bit each).
- Stored as tensor $X \in \mathbb{R}^{H \times W \times 3}$.
- Examples:
 - $(255, 0, 0)$ = red
 - $(0, 255, 0)$ = green
 - $(0, 0, 255)$ = blue
 - $(255, 255, 255)$ = white

Grayscale vs. RGB Images (How Computers Represent Them)

Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.
- Typical 8-bit encoding: values in $[0, 255]$
 - 0 = black
 - 255 = white
 - middle values = shades of gray
- Example: 128 $\rightarrow$ mid-gray

RGB (3 channels)

- Each pixel stores a **3-tuple** (R, G, B) .
- Each channel has values in $[0, 255]$ (8-bit each).
- Stored as tensor $X \in \mathbb{R}^{H \times W \times 3}$.
- Examples:
 - $(255, 0, 0)$ = red
 - $(0, 255, 0)$ = green
 - $(0, 0, 255)$ = blue
 - $(255, 255, 255)$ = white

Grayscale vs. RGB Images (How Computers Represent Them)

Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.
- Typical 8-bit encoding: values in $[0, 255]$
 - 0 = black
 - 255 = white
 - middle values = shades of gray
- Example: 128 $\rightarrow$ mid-gray

How computers “read” an image:

- Image files are decoded into arrays indexed by (i, j) (and channel).

RGB (3 channels)

- Each pixel stores a **3-tuple** (R, G, B) .
- Each channel has values in $[0, 255]$ (8-bit each).
- Stored as tensor $X \in \mathbb{R}^{H \times W \times 3}$.
- Examples:
 - $(255, 0, 0)$ = red
 - $(0, 255, 0)$ = green
 - $(0, 0, 255)$ = blue
 - $(255, 255, 255)$ = white

Grayscale vs. RGB Images (How Computers Represent Them)

Grayscale (1 channel)

- Each pixel stores a single **intensity** value (black $\rightarrow$ white).
- Stored as a matrix $I \in \mathbb{R}^{H \times W}$.
- Typical 8-bit encoding: values in $[0, 255]$
 - 0 = black
 - 255 = white
 - middle values = shades of gray
- Example: 128 $\rightarrow$ mid-gray

How computers “read” an image:

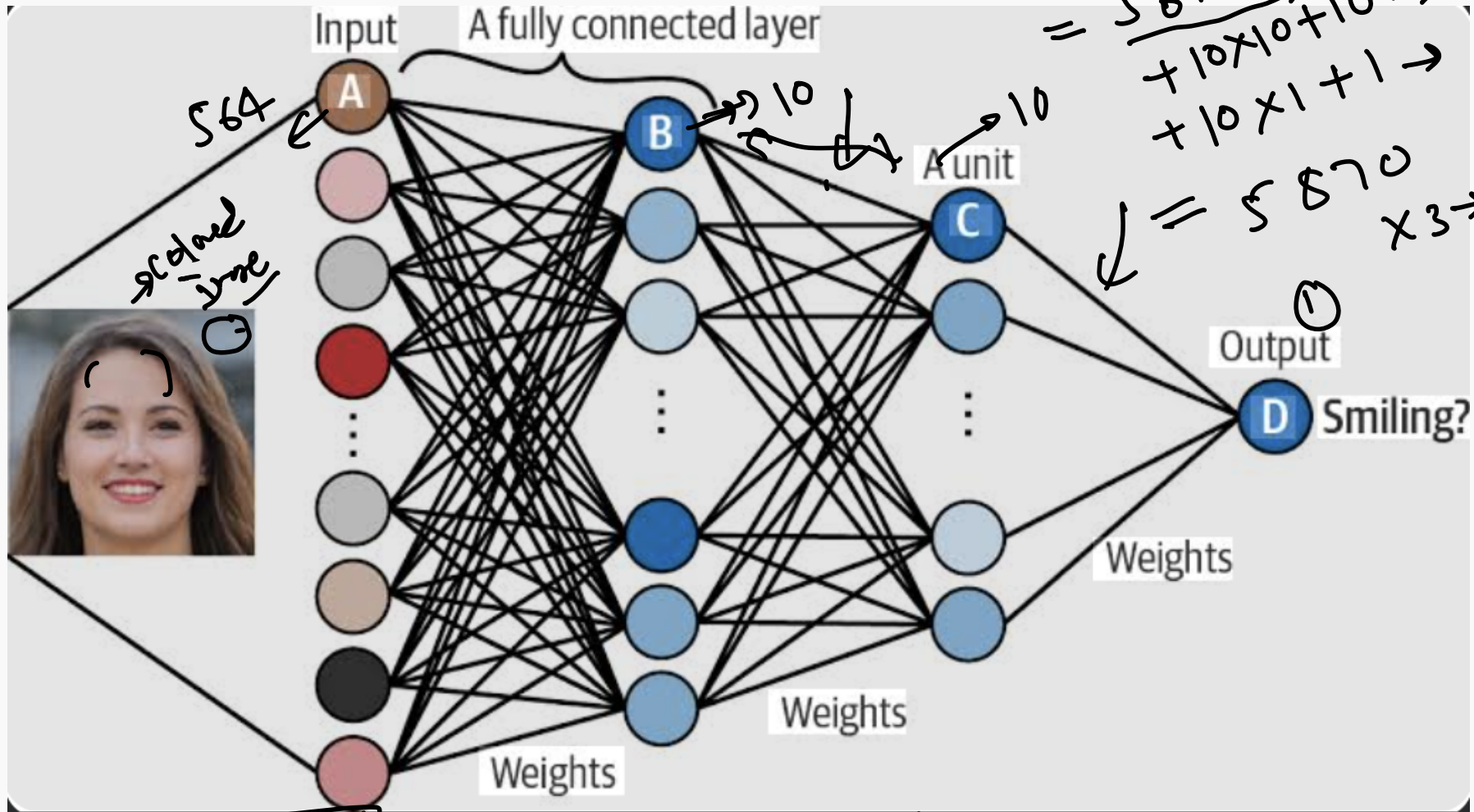
- Image files are decoded into arrays indexed by (i, j) (and channel).
- These pixel values are just numbers used as input features in ML/CNNs.

RGB (3 channels)

- Each pixel stores a **3-tuple** (R, G, B) .
- Each channel has values in $[0, 255]$ (8-bit each).
- Stored as tensor $X \in \mathbb{R}^{H \times W \times 3}$.
- Examples:
 - $(255, 0, 0)$ = red
 - $(0, 255, 0)$ = green
 - $(0, 0, 255)$ = blue
 - $(\underbrace{255, 255, 255}_{RGB}) = \underline{\text{white}}$

MLPs for Image Data: Just Flatten an Image and use an MLP

↓
NN(s) that we
have learned so far



$$= \frac{554 + 10 + 10}{+ 10 \times 10 + 10} \rightarrow 5650$$

$$+ 10 \times 1 + 1 \rightarrow 11$$

$$= 5870$$

$$\times 3 \rightarrow \text{ReLU}$$

gray-scale

564 pixels

$28 \times 28 = 564$

x_1
 x_2
 $\vdots$

y_{564} | 564×1

dim of input

$D = 564 \Rightarrow$

Why not Just Flatten an Image and use an MLP? (Parameter Explosion) II

Example: RGB image $64 \times 64 \times 3$ flattened to $d = 12,288$.

- Dense layer with $m = 1,024$ hidden units has:

$$\# \text{params} = d m + m = 12,288 \cdot 1,024 + 1,024 \approx 12.6 \text{ million}$$

- **Scales badly:** increasing resolution (e.g., 224×224) makes d huge $\Rightarrow$ parameters and compute explode.

Why not Just Flatten an Image and use an MLP? (Parameter Explosion) II

Example: RGB image $64 \times 64 \times 3$ flattened to $d = 12,288$.

- Dense layer with $m = 1,024$ hidden units has:

$$\# \text{params} = d m + m = 12,288 \cdot 1,024 + 1,024 \approx 12.6 \text{ million}$$

- **Scales badly:** increasing resolution (e.g., 224×224) makes d huge $\Rightarrow$ parameters and compute explode.
- **Overfitting risk:** too many degrees of freedom unless you have massive labeled data.

Why not Just Flatten an Image and use an MLP? (Parameter Explosion) II

Example: RGB image $64 \times 64 \times 3$ flattened to $d = 12,288$.

- Dense layer with $m = 1,024$ hidden units has:

$$\# \text{params} = d m + m = 12,288 \cdot 1,024 + 1,024 \approx 12.6 \text{ million}$$

- **Scales badly:** increasing resolution (e.g., 224×224) makes d huge $\Rightarrow$ parameters and compute explode.
- **Overfitting risk:** too many degrees of freedom unless you have massive labeled data.
- **No weight sharing:** the same edge/texture at different locations requires separate parameters.

Why not Just Flatten an Image and use an MLP? (Parameter Explosion) II

Example: RGB image $64 \times 64 \times 3$ flattened to $d = 12,288$.

- Dense layer with $m = 1,024$ hidden units has:

$$\# \text{params} = d m + m = 12,288 \cdot 1,024 + 1,024 \approx 12.6 \text{ million}$$

- **Scales badly:** increasing resolution (e.g., 224×224) makes d huge $\Rightarrow$ parameters and compute explode.
- **Overfitting risk:** too many degrees of freedom unless you have massive labeled data.
- **No weight sharing:** the same edge/texture at different locations requires separate parameters.
- **No explicit locality:** every neuron sees all pixels, so it does not prefer local patterns.

Why not Just Flatten an Image and use an MLP? (Parameter Explosion) II

Example: RGB image $64 \times 64 \times 3$ flattened to $d = 12,288$.

- Dense layer with $m = 1,024$ hidden units has:

$$\# \text{params} = d m + m = 12,288 \cdot 1,024 + 1,024 \approx 12.6 \text{ million}$$

- **Scales badly:** increasing resolution (e.g., 224×224) makes d huge $\Rightarrow$ parameters and compute explode.
- **Overfitting risk:** too many degrees of freedom unless you have massive labeled data.
- **No weight sharing:** the same edge/texture at different locations requires separate parameters.
- **No explicit locality:** every neuron sees all pixels, so it does not prefer local patterns.
- **Poor inductive bias for images:** translation structure is not built-in, so generalization is weaker.

Why not Just Flatten an Image and use an MLP? (Parameter Explosion) II

Example: RGB image $64 \times 64 \times 3$ flattened to $d = 12,288$.

- Dense layer with $m = 1,024$ hidden units has:

$$\# \text{params} = d m + m = 12,288 \cdot 1,024 + 1,024 \approx 12.6 \text{ million}$$

- **Scales badly:** increasing resolution (e.g., 224×224) makes d huge $\Rightarrow$ parameters and compute explode.
- **Overfitting risk:** too many degrees of freedom unless you have massive labeled data.
- **No weight sharing:** the same edge/texture at different locations requires separate parameters.
- **No explicit locality:** every neuron sees all pixels, so it does not prefer local patterns.
- **Poor inductive bias for images:** translation structure is not built-in, so generalization is weaker.
- **Memory/latency:** large dense matrices are expensive (GPU/TPU memory + bandwidth).

What Structure do Images Have That MLPs Ignore?

- **Locality:** nearby pixels are more related than far-away pixels.
- **Translation:** the same pattern (edge/texture) can appear anywhere in the image.
- **Stationarity:** low-level statistics (edges/textures) are similar across locations.
- **Compositionality:** edges $\rightarrow$ corners $\rightarrow$ parts $\rightarrow$ objects.
- **Multi-scale structure:** relevant patterns exist at multiple spatial scales.

What Structure do Images Have That MLPs Ignore?

- **Locality**: nearby pixels are more related than far-away pixels.
- **Translation**: the same pattern (edge/texture) can appear anywhere in the image.
- **Stationarity**: low-level statistics (edges/textures) are similar across locations.
- **Compositionality**: edges $\rightarrow$ corners $\rightarrow$ parts $\rightarrow$ objects.
- **Multi-scale structure**: relevant patterns exist at multiple spatial scales.

CNNs encode these as **inductive biases** via:

- **Local connectivity** (small receptive fields),
- **Weight sharing** (same kernel everywhere),
- **Downsampling** (pooling/striding) for multi-scale features,
- **Hierarchy**: deeper layers aggregate simpler features into more complex ones.

Note: Convolutions are **translation equivariant** (shifting the input shifts the feature map). Pooling and global averaging help build **approximate invariance**.

Convolutional Neural Networks (CNNs)

Agenda

2. Convolutional Neural Networks (CNNs)

2.1 From MLP to CNN

2.2 Core Components of CNNs

Convolution

Pooling

Fully Connected Layers

2.3 Full CNN Pipeline

2.4 Training CNNs

2.5 CNNs Lab

From MLPs to CNNs: the key Idea

- A **CNN** is a neural network designed for grid-structured data (images, spectrograms, time-series).
- Replace dense connections with **convolutions**: a small kernel slides across the input.
- **Weight sharing** means the number of parameters does *not* grow with image width/height.

From MLPs to CNNs: the key Idea

- A **CNN** is a neural network designed for grid-structured data (images, spectrograms, time-series).
- Replace dense connections with **convolutions**: a small kernel slides across the input.
- **Weight sharing** means the number of parameters does *not* grow with image width/height.

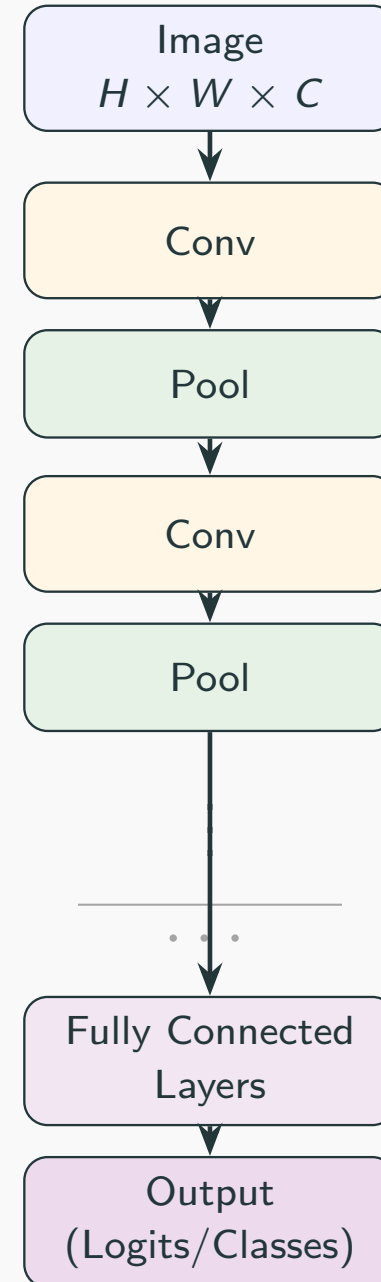
Parameter comparison (typical):

- Dense layer (flattened input): $\# \text{params} \approx (H W C) \cdot m$.
- Conv layer: $\# \text{params} = K (k k C + 1)$ (independent of H, W).
- Benefits: **fewer parameters**, **translation equivariance**, and **hierarchical feature learning**.
- Typical pipeline: **Conv** $\rightarrow$ **Nonlinearity** $\rightarrow$ (optional) **Normalization** $\rightarrow$ (optional) **Pooling/Stride** $\rightarrow$ repeat.
- End: **global average pooling** or **fully connected** head for predictions.

A Standard CNN Block (and why it Works)

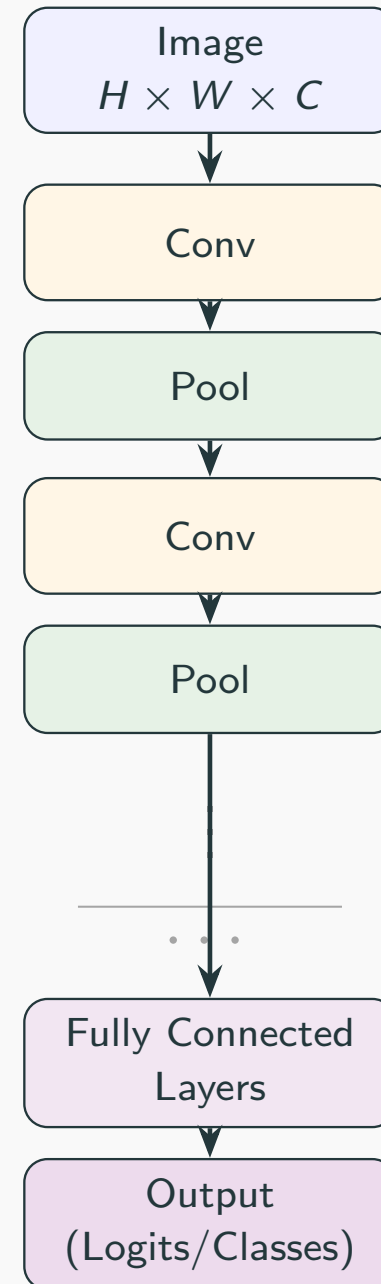
- **Convolution layer:** learn local feature detectors (edges, textures, parts).

⇒ 1) convolutional Layer
2) pooling layers
3) full flattened layers



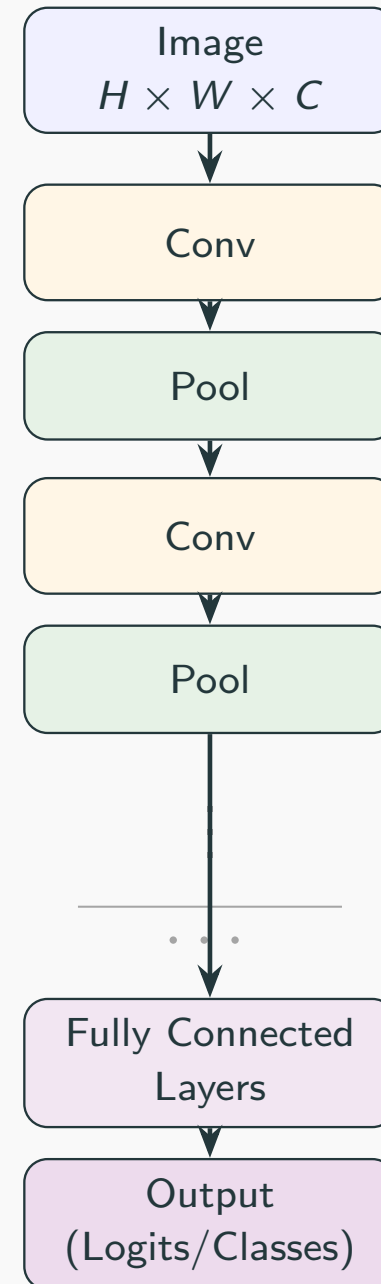
A Standard CNN Block (and why it Works)

- **Convolution layer:** learn local feature detectors (edges, textures, parts).
- **Activation:** nonlinearity (usually ReLU).



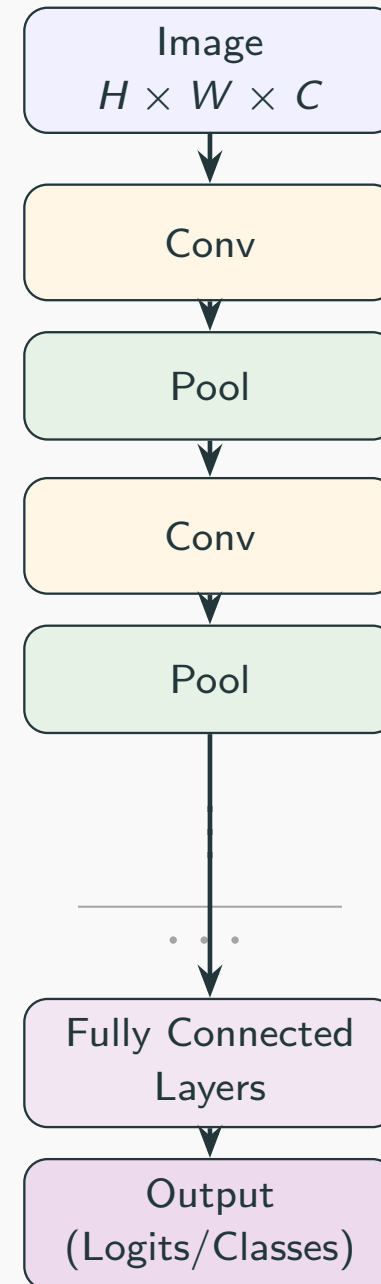
A Standard CNN Block (and why it Works)

- **Convolution layer:** learn local feature detectors (edges, textures, parts).
- **Activation:** nonlinearity (usually ReLU).
- **Normalization:** It stabilizes activations and gradients (optional but common).



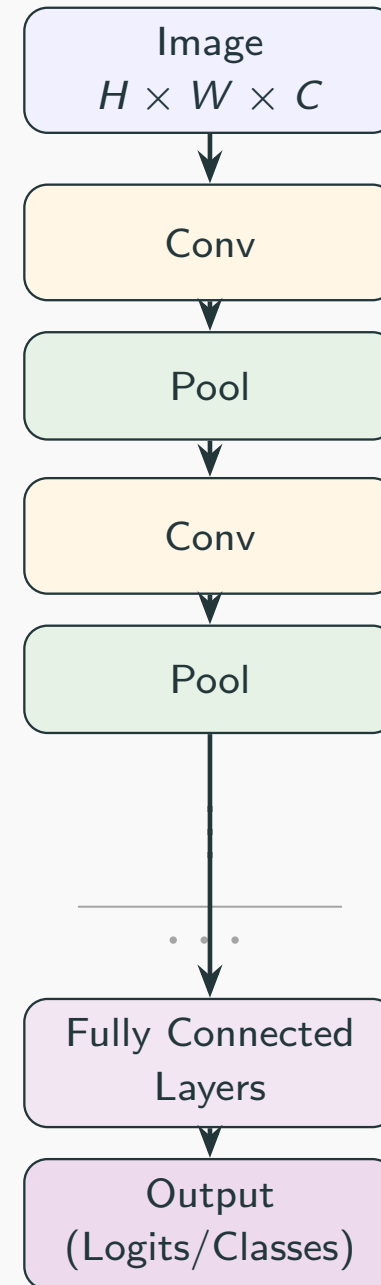
A Standard CNN Block (and why it Works)

- **Convolution layer:** learn local feature detectors (edges, textures, parts).
- **Activation:** nonlinearity (usually ReLU).
- **Normalization:** It stabilizes activations and gradients (optional but common).
- **Pooling / Stride layer:** downsample to reduce compute.



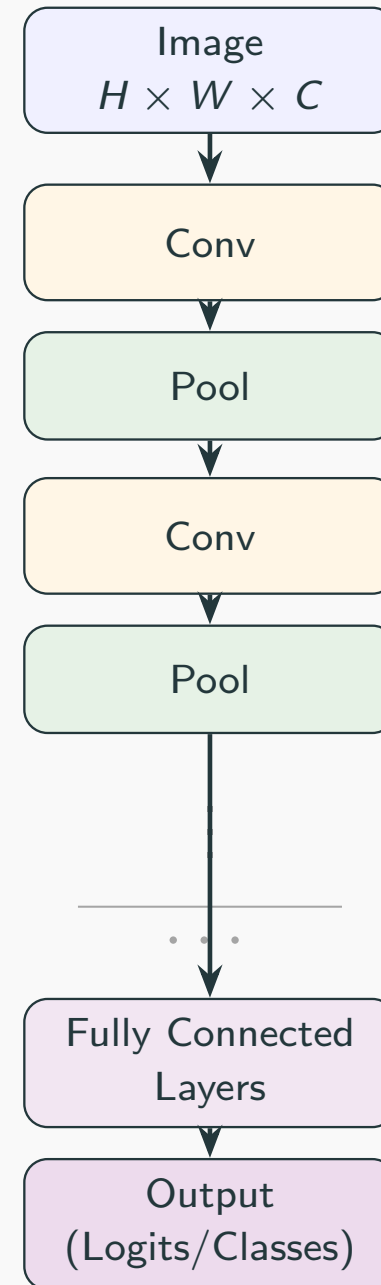
A Standard CNN Block (and why it Works)

- **Convolution layer:** learn local feature detectors (edges, textures, parts).
- **Activation:** nonlinearity (usually ReLU).
- **Normalization:** It stabilizes activations and gradients (optional but common).
- **Pooling / Stride layer:** downsample to reduce compute.
- **Skip Connections** (ResNets): improve gradient flow and enable deeper networks.



A Standard CNN Block (and why it Works)

- 1)
 - **Convolution layer:** learn local feature detectors (edges, textures, parts).
 - **Activation:** nonlinearity (usually ReLU).
 - **Normalization:** It stabilizes activations and gradients (optional but common).
- 2)
 - **Pooling / Stride layer:** downsample to reduce compute.
 - **Skip Connections** (ResNets): improve gradient flow and enable deeper networks.
- 3)
 - **Fully Connected Layers:** fully-connected (FC) or global pooling → classifier.



Agenda

2. Convolutional Neural Networks (CNNs)

2.1 From MLP to CNN

2.2 Core Components of CNNs

Convolution

Pooling

Fully Connected Layers

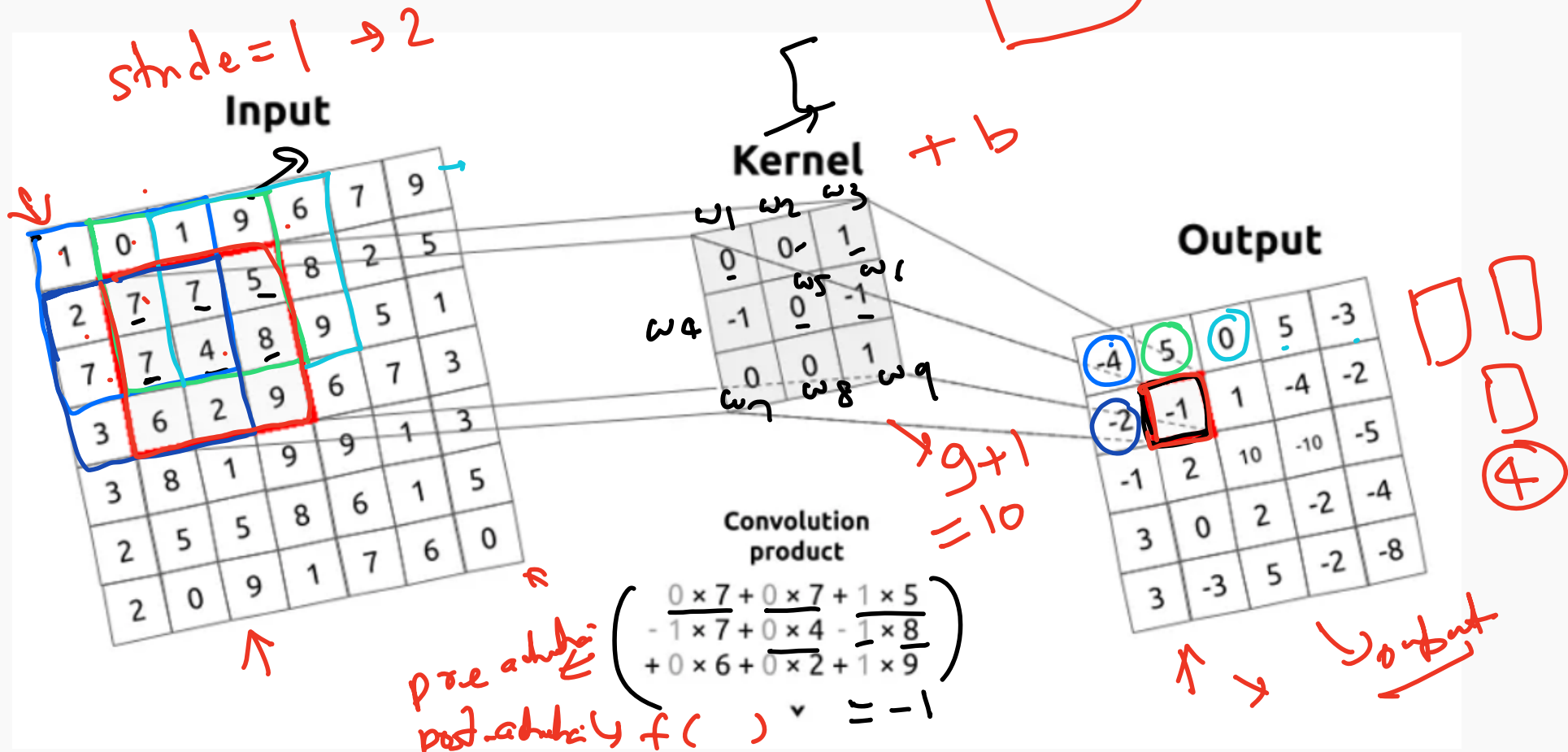
2.3 Full CNN Pipeline

2.4 Training CNNs

2.5 CNNs Lab

What is Convolution? (Intuition)

- A small filter scans across the image
- Detects patterns like edges, textures
- Same filter applied everywhere



Convolution Operation

$$Y[i, j] = \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} W[u, v] X[i + u, j + v]$$


- W : filter (kernel), learned during training
- X : input image (or previous feature map)
- Y : output feature map (one map per filter)
- Intuition: each output pixel is a *weighted sum* of a local patch

Learn more from: <https://developer.nvidia.com/discover/convolution>



Channels (Depth)

- A **channel** is one *feature map* along the depth dimension of the data tensor.

of kernels

$[]_{3 \times 3}$

$\Rightarrow$ 4 channels

$[]_{4 \times 4}$

$[]_{5 \times 5}$

$[]_{2 \times 2}$

Channels (Depth)

- A **channel** is one *feature map* along the depth dimension of the data tensor.
- In general, data can be represented as:

$$X \in \mathbb{R}^{d_1 \times d_2 \times \dots \times d_k \times C}$$

where C denotes the number of channels, and each channel is:

$$X[:, :, \dots, :, c]$$

Channels (Depth)

- A **channel** is one *feature map* along the depth dimension of the data tensor.
- In general, data can be represented as:

$$X \in \mathbb{R}^{d_1 \times d_2 \times \dots \times d_k \times C}$$

where C denotes the number of channels, and each channel is:

$$X[:, :, \dots, :, c]$$

- **Interpretation:** A channel captures one type of information or feature across the spatial/temporal dimensions.

Channels (Depth)

- A **channel** is one *feature map* along the depth dimension of the data tensor.
- In general, data can be represented as:

$$X \in \mathbb{R}^{d_1 \times d_2 \times \dots \times d_k \times C}$$

where C denotes the number of channels, and each channel is:

$$X[:, :, \dots, :, c]$$

- **Interpretation:** A channel captures one type of information or feature across the spatial/temporal dimensions.

- **Special cases:**

- **1D data (signals, time series):** $X \in \mathbb{R}^{L \times C}$
Channel = 1D signal
- **2D data (images):** $X \in \mathbb{R}^{H \times W \times C}$
Channel = 2D map (e.g., RGB planes)
- **3D data (videos, volumetric):** $X \in \mathbb{R}^{D \times H \times W \times C}$
Channel = 3D volume

Filters (Kernels): How Channels are Produced i

- A **filter/kernel** detects local patterns by sliding over the input
- **Key idea:**
 - Each filter spans **all input channels**
 - **One filter $\Rightarrow$ one output channel**
 - **K filters $\Rightarrow K$ output channels**
- **1D Convolution (Time Series)**
 - Input: $X \in \mathbb{R}^{L \times C}$
 - One filter: $W^{(m)} \in \mathbb{R}^{k \times C}$
 - Output (one channel):

$$Y[:, m] = \sum_{c=1}^C W^{(m)}[:, c] * X[:, c]$$

- Using K filters $\Rightarrow Y \in \mathbb{R}^{L' \times K}$

Filters (Kernels): How Channels are Produced ii

- **2D Convolution (Images)**

- Input: $X \in \mathbb{R}^{H \times W \times C}$
- One filter: $W^{(m)} \in \mathbb{R}^{k \times k \times C}$
- Output (one channel):

$$Y[:, :, m] = \sum_{c=1}^C W^{(m)}[:, :, c] * X[:, :, c]$$

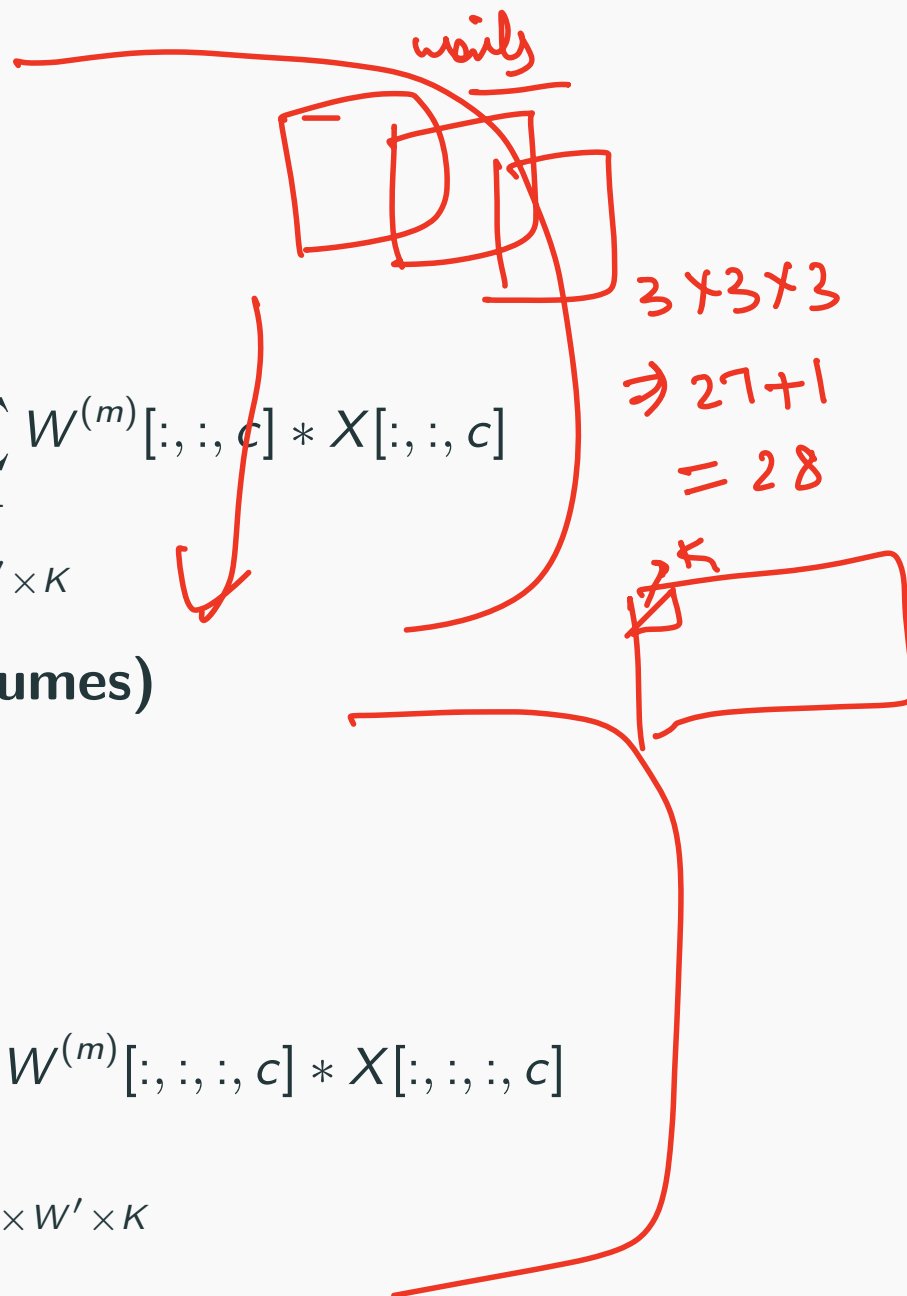
- Using K filters $\Rightarrow Y \in \mathbb{R}^{H' \times W' \times K}$

- **3D Convolution (Videos / Volumes)**

- Input: $X \in \mathbb{R}^{D \times H \times W \times C}$
- One filter: $W^{(m)} \in \mathbb{R}^{k \times k \times k \times C}$
- Output (one channel):

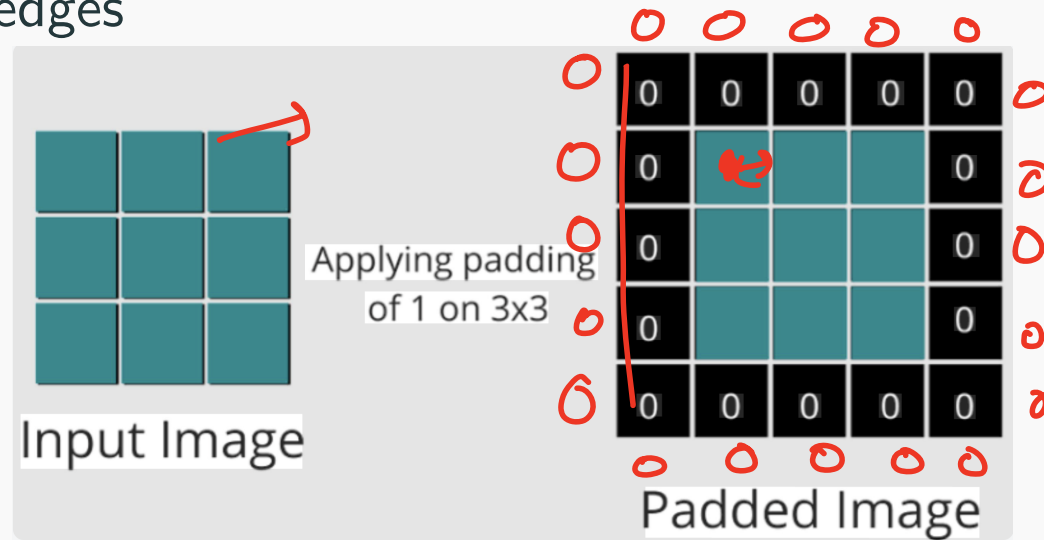
$$Y[:, :, :, m] = \sum_{c=1}^C W^{(m)}[:, :, :, c] * X[:, :, :, c]$$

- Using K filters $\Rightarrow Y \in \mathbb{R}^{D' \times H' \times W' \times K}$



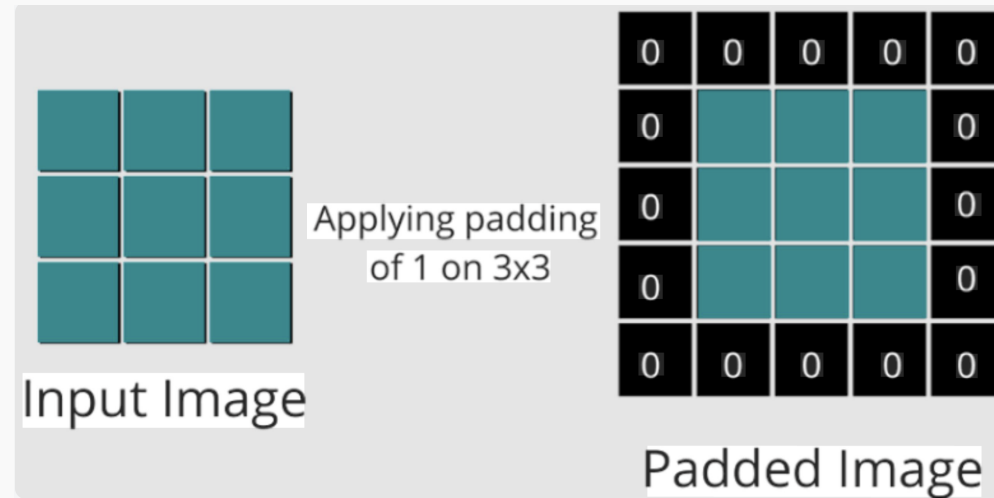
Padding in CNNs: When and Why?

- **Padding** adds border pixels (usually zeros) so convolution can be applied near edges



Padding in CNNs: When and Why?

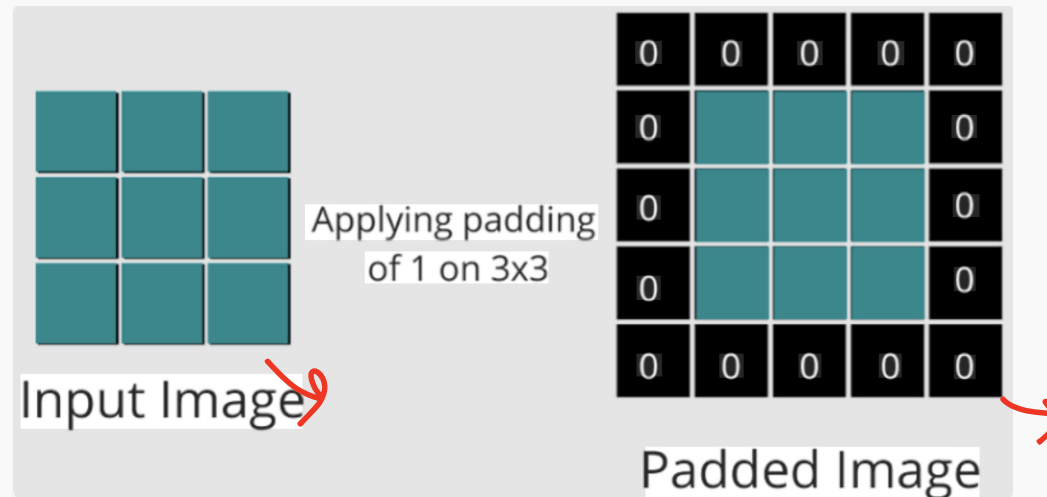
- **Padding** adds border pixels (usually zeros) so convolution can be applied near edges



- Why it matters:
 - avoids shrinking the feature map after every conv layer
 - preserves information at image borders (otherwise borders are seen fewer times)
 - helps keep shapes aligned across layers (useful in deep CNNs / skip connections)

Padding in CNNs: When and Why?

- **Padding** adds border pixels (usually zeros) so convolution can be applied near edges



- Why it matters:
 - avoids shrinking the feature map after every conv layer
 - preserves information at image borders (otherwise borders are seen fewer times)
 - helps keep shapes aligned across layers (useful in deep CNNs / skip connections)
- Two types of padding:
 - Valid padding: $P = 0$ (no padding) $\Rightarrow$ output shrinks
 - Same padding: choose P to roughly preserve size $\rightarrow$ 3/3

Activation Functions in CNNs: Why and When?

- Why do we need them?
 - Without activation functions, the network is just a **linear model**
 - Stacking linear layers $\Rightarrow$ still linear (no extra power)
 - Activations introduce **non-linearity** $\Rightarrow$ can learn complex patterns

Activation Functions in CNNs: Why and When?

- Why do we need them?
 - Without activation functions, the network is just a **linear model**
 - Stacking linear layers $\Rightarrow$ still linear (no extra power)
 - Activations introduce **non-linearity** $\Rightarrow$ can learn complex patterns
- ReLU (Rectified Linear Unit)

$$\text{ReLU}(x) = \max(0, x) \rightarrow \sigma_h$$

- Very simple and computationally efficient
- Avoids vanishing gradients (unlike sigmoid/tanh)

$$\nabla L \approx 0$$

$$\omega_{t+1} = \omega_t - \eta \frac{\partial L}{\partial \omega_t} = 0$$

Activation Functions in CNNs: Why and When?

- Why do we need them?

- Without activation functions, the network is just a **linear model**
- Stacking linear layers $\Rightarrow$ still linear (no extra power)
- Activations introduce **non-linearity** $\Rightarrow$ can learn complex patterns

- ReLU (Rectified Linear Unit)

$$\text{ReLU}(x) = \max(0, x)$$

- Very simple and computationally efficient
- Avoids vanishing gradients (unlike sigmoid/tanh)

- Leaky ReLU

$$\text{LeakyReLU}(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}, \quad \alpha > 0 \quad \text{is a small constant}$$

- Fixes “dead neuron” problem in ReLU
- Allows small gradient when $x < 0$

- When are they applied?

- Applied **after every convolution layer**: Typically Conv $\rightarrow$ ReLU

Key Hyperparameters in Convolution

- ^{filter} Kernel size ($k_h \times k_w$): size of filter window (e.g., 3×3)
- Stride (S): step size when sliding the filter (e.g., $S = 2$ downsamples)
- Padding (P): zeros added at borders to control output size
 - Valid: no padding ($P = 0$)
 - Same: output size same as input
- Number of filters (K):
 - Determines number of output channels
- (Optional) **Dilation**:
 - Expands receptive field without increasing parameters

Key Hyperparameters in Convolution

- **Kernel size** ($k_h \times k_w$): size of filter window (e.g., 3×3)
- **Stride** (S): step size when sliding the filter (e.g., $S = 2$ downsamples)
- **Padding** (P): zeros added at borders to control output size
 - **Valid**: no padding ($P = 0$)
 - **Same**: output size same as input
- **Number of filters** (K):
 - Determines number of output channels
- **(Optional) Dilation**:
 - Expands receptive field without increasing parameters

Output Size Formula

$$H' = \left\lfloor \frac{H + 2P - k_h}{S} \right\rfloor + 1, \quad W' = \left\lfloor \frac{W + 2P - k_w}{S} \right\rfloor + 1$$

$$\text{Output: } X' \in \mathbb{R}^{H' \times W' \times K}$$

- Example: $H = W = 32$, $k = 3$, $S = 1$, $P = 1 \Rightarrow H' = W' = 32$ (same)

Why Pooling?

- **Reduces spatial size:**
 - From $H \times W \rightarrow H' \times W'$
 - Reduces computation and memory
- **Provides translation invariance:**
 - Small shifts in input do not change output much
- **Increases receptive field:**
 - Each pooled value summarizes a larger region
- **Reduces overfitting:**
 - Acts as a form of downsampling / regularization

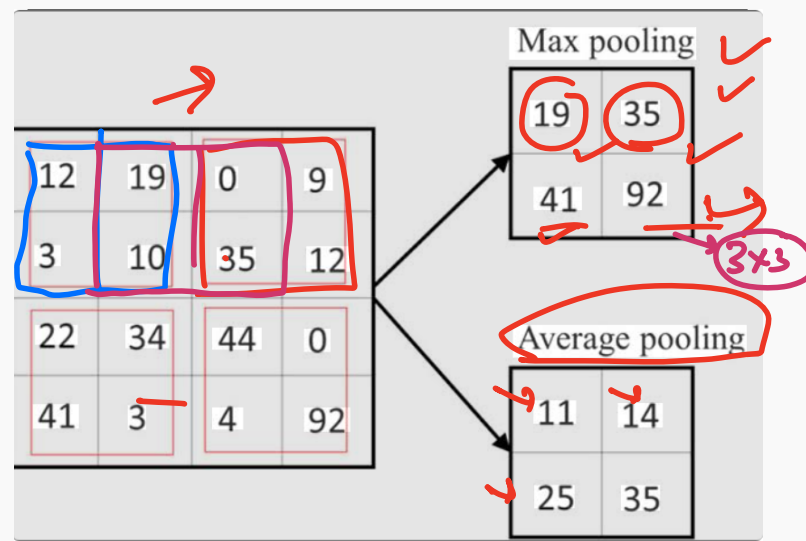
Pooling Types (and When to Use Them)

- Max Pooling

- Takes the maximum value in each window
- **Important when:** you care about *whether a feature exists* (edge/texture presence), and want robustness to small shifts
- Common in early CNNs and in feature-extraction backbones

- Average Pooling

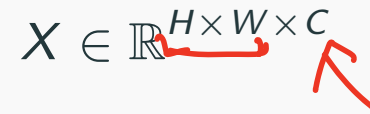
- Takes the mean value in each window
- **Important when:** you want *smoother* / less noisy activations (e.g., low-texture regions, denoising-like effects)



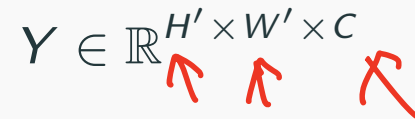
⇒ No parameters

How Pooling Works

- Applied **independently** on each channel
- For input:

$$X \in \mathbb{R}^{H \times W \times C}$$


- A window of size $k \times k$ slides with stride s
- Output:

$$Y \in \mathbb{R}^{H' \times W' \times C}$$


- **Important:**
 - Number of channels C remains unchanged
 - Only spatial dimensions are reduced

Fully Connected (Dense) Layers i

- A **Fully Connected (FC)** layer connects every input unit to every output unit

- **Role in CNNs:**

- Placed near the **end of the network**
- Combines all extracted features **globally**
- Performs **final prediction / classification**

- **Input:**

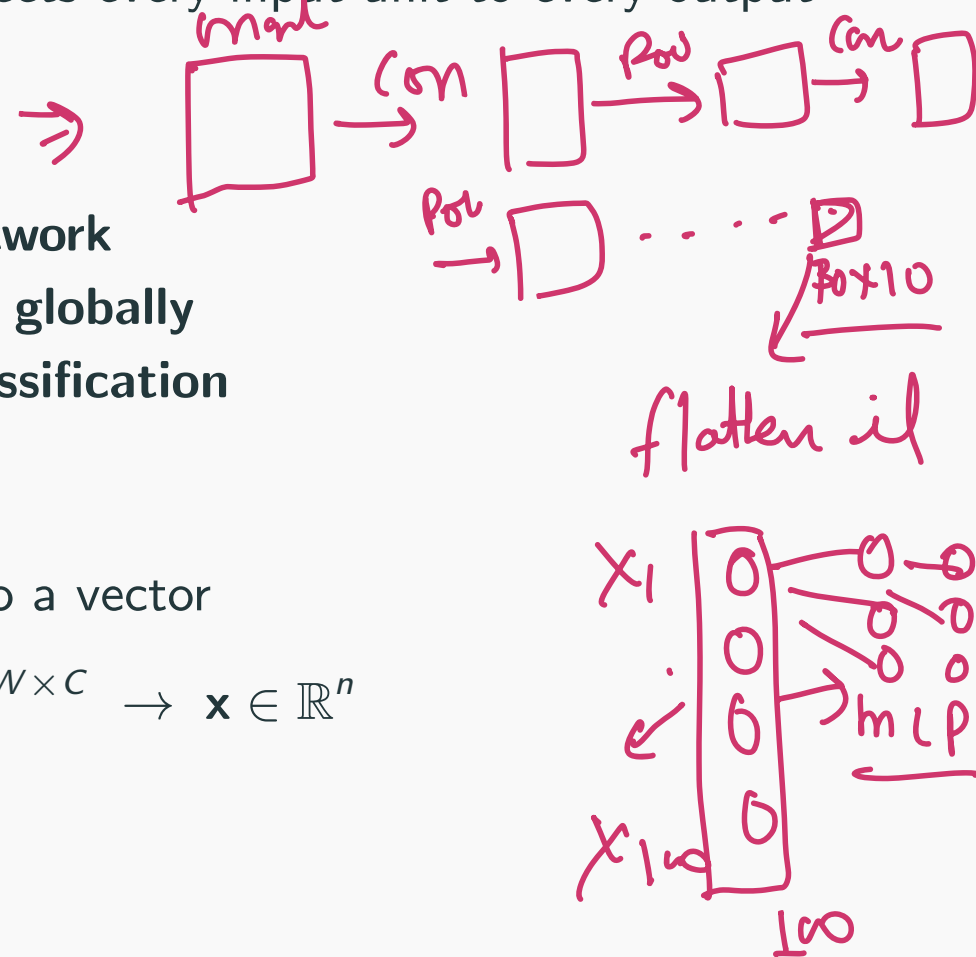
- Feature maps are **flattened** into a vector

$$X \in \mathbb{R}^{H \times W \times C} \rightarrow \mathbf{x} \in \mathbb{R}^n$$

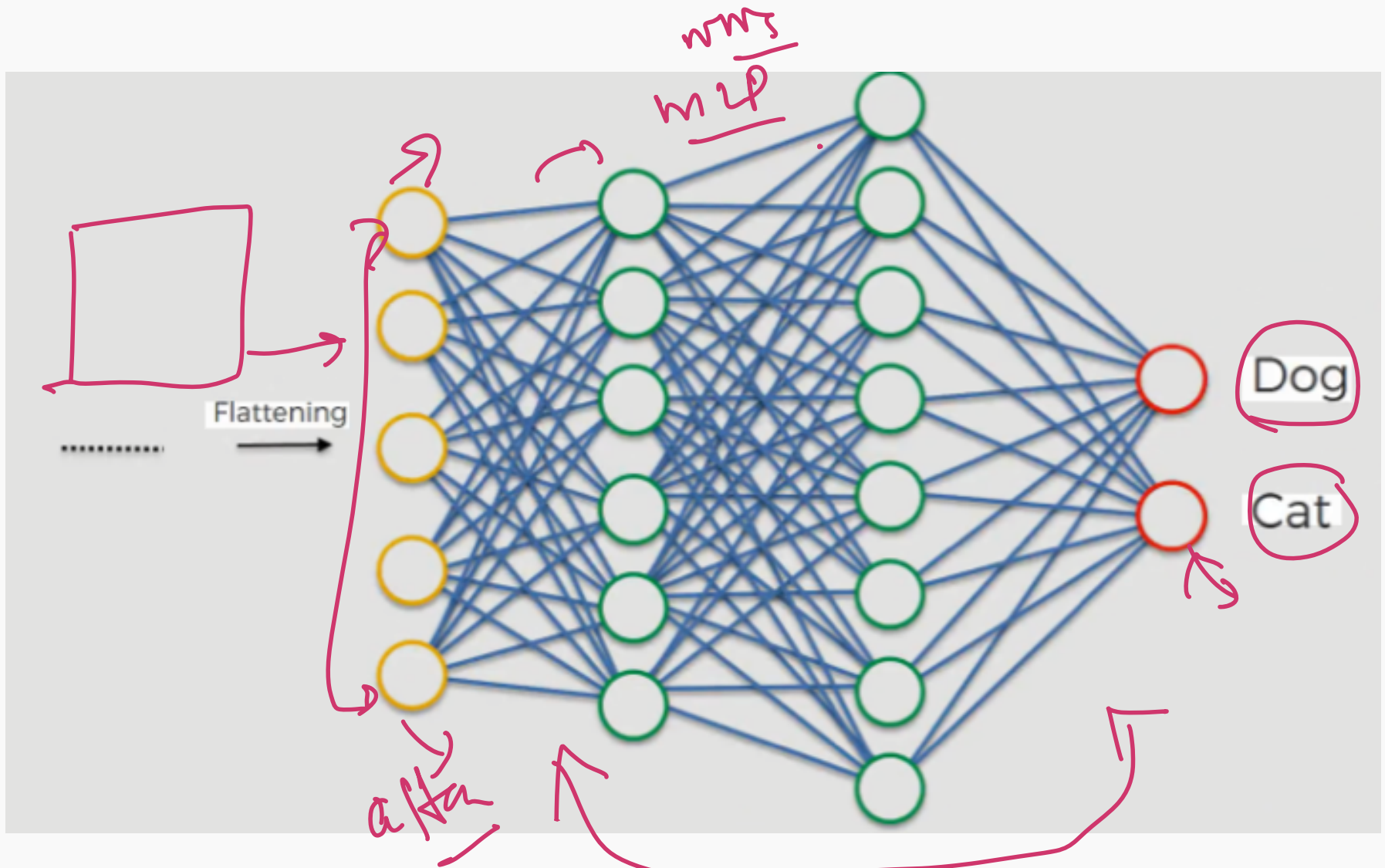
where $n = HWC$

- **Interpretation:**

- Learns **global combinations** of features
- Acts like a standard **neural network layer**
- Has **large number of parameters** (can cause overfitting)



Fully Connected Layers ii



Agenda

2. Convolutional Neural Networks (CNNs)

2.1 From MLP to CNN

2.2 Core Components of CNNs

Convolution

Pooling

Fully Connected Layers

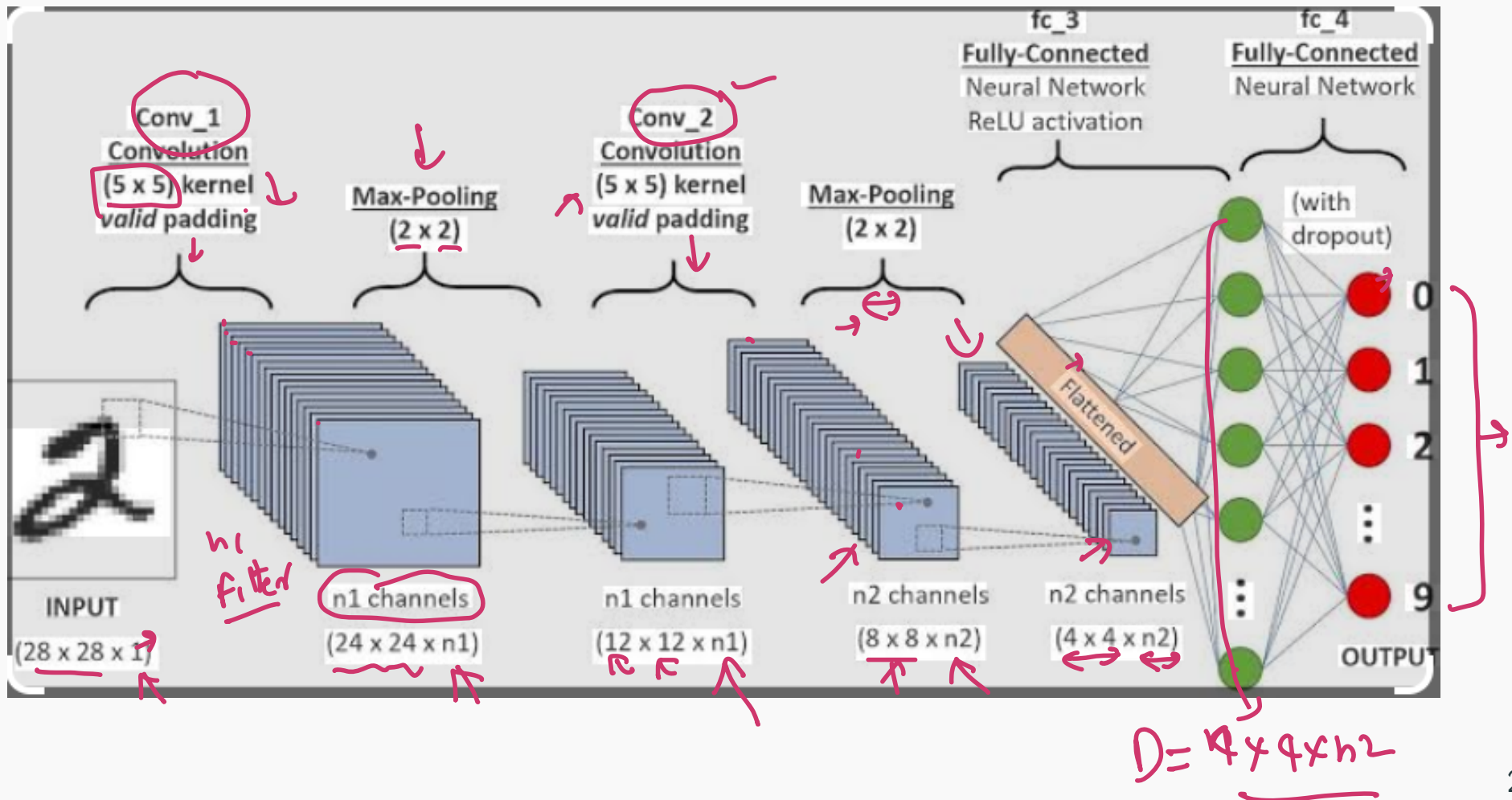
2.3 Full CNN Pipeline

2.4 Training CNNs

2.5 CNNs Lab

One Example of CNN Architecture

Input $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ Pool
 $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ Pool
 $\rightarrow$ Fully Connected $\rightarrow$ Softmax (for classification)



Agenda

2. Convolutional Neural Networks (CNNs)

2.1 From MLP to CNN

2.2 Core Components of CNNs

Convolution

Pooling

Fully Connected Layers

2.3 Full CNN Pipeline

2.4 Training CNNs

2.5 CNNs Lab

Training CNNs: Overall Pipeline

- Training consists of repeatedly updating parameters to minimize a loss function

Training CNNs: Overall Pipeline

- Training consists of repeatedly updating parameters to minimize a loss function

- Step 1: Forward Pass *(forward propagation)*
 - Input passes through layers (Conv, Activation, Pooling, FC)
 - Produces prediction:

$$\hat{y} = f(\mathbf{x}; \theta) \rightarrow$$

Training CNNs: Overall Pipeline

- Training consists of repeatedly updating parameters to minimize a loss function

- **Step 1: Forward Pass**

- Input passes through layers (Conv, Activation, Pooling, FC)
- Produces prediction:

$$\hat{y} = f(\mathbf{x}; \theta)$$



- **Step 2: Compute Loss**

- Measures error between prediction and true label

$$\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})$$

- Example (classification): Cross-entropy loss

Training CNNs: Overall Pipeline

- Training consists of repeatedly updating parameters to minimize a loss function

- **Step 1: Forward Pass**

- Input passes through layers (Conv, Activation, Pooling, FC)
- Produces prediction:

$$\hat{y} = f(\mathbf{x}; \theta)$$

- **Step 2: Compute Loss**

- Measures error between prediction and true label

$$\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})$$

- Example (classification): Cross-entropy loss

- **Step 3: Backpropagation**

- Compute gradients:

$$\left(\frac{\partial \mathcal{L}}{\partial \theta} \right)$$

- Uses chain rule layer-by-layer

Training CNNs: Overall Pipeline

- Training consists of repeatedly updating parameters to minimize a loss function

- **Step 1: Forward Pass**

- Input passes through layers (Conv, Activation, Pooling, FC)
- Produces prediction:

$$\hat{y} = f(\mathbf{x}; \theta)$$

- **Step 2: Compute Loss**

- Measures error between prediction and true label

$$\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}}) \quad \checkmark$$

- Example (classification): Cross-entropy loss

- **Step 3: Backpropagation**

- Compute gradients:

$$\frac{\partial \mathcal{L}}{\partial \theta} \rightarrow w^{(1)}$$

- Uses chain rule layer-by-layer

- **Step 4: Parameter Update**

- Gradient descent:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$

Training CNNs: Key Details

- What is learned?
 - Convolution filters (kernels)
 - Fully connected weights
 - Bias terms

Training CNNs: Key Details

- **What is learned?**
 - Convolution filters (kernels)
 - Fully connected weights
 - Bias terms
- **Backpropagation in CNNs**
 - Gradients flow through:
 - Convolution layers (shared weights)
 - Activation functions (e.g., ReLU)
 - Pooling layers (routes gradients)

Training CNNs: Key Details

- **What is learned?**
 - Convolution filters (kernels)
 - Fully connected weights
 - Bias terms
- **Backpropagation in CNNs**
 - Gradients flow through:
 - Convolution layers (shared weights)
 - Activation functions (e.g., ReLU)
 - Pooling layers (routes gradients)
- **Mini-batch Training**
 - Use small batches of data for efficiency
 - Stabilizes gradient estimates

Training CNNs: Key Details

- **What is learned?**
 - Convolution filters (kernels)
 - Fully connected weights
 - Bias terms
- **Backpropagation in CNNs**
 - Gradients flow through:
 - Convolution layers (shared weights)
 - Activation functions (e.g., ReLU)
 - Pooling layers (routes gradients)
- **Mini-batch Training**
 - Use small batches of data for efficiency
 - Stabilizes gradient estimates
- **Optimization Algorithms**
 - SGD, Momentum, Adam

Training CNNs: Key Details

- **What is learned?**
 - Convolution filters (kernels)
 - Fully connected weights
 - Bias terms
- **Backpropagation in CNNs**
 - Gradients flow through:
 - Convolution layers (shared weights)
 - Activation functions (e.g., ReLU)
 - Pooling layers (routes gradients)
- **Mini-batch Training**
 - Use small batches of data for efficiency
 - Stabilizes gradient estimates
- **Optimization Algorithms**
 - SGD, Momentum, Adam
- **Important Practices**
 - Learning rate tuning
 - Regularization (dropout, weight decay)
 - Batch normalization (faster convergence)

Training CNNs: Key Details

- **What is learned?**
 - Convolution filters (kernels)
 - Fully connected weights
 - Bias terms
- **Backpropagation in CNNs**
 - Gradients flow through:
 - Convolution layers (shared weights)
 - Activation functions (e.g., ReLU)
 - Pooling layers (routes gradients)
- **Mini-batch Training**
 - Use small batches of data for efficiency
 - Stabilizes gradient estimates
- **Optimization Algorithms**
 - SGD, Momentum, Adam
- **Important Practices**
 - Learning rate tuning
 - Regularization (dropout, weight decay)
 - Batch normalization (faster convergence)

Agenda

2. Convolutional Neural Networks (CNNs)

2.1 From MLP to CNN

2.2 Core Components of CNNs

Convolution

Pooling

Fully Connected Layers

2.3 Full CNN Pipeline

2.4 Training CNNs

2.5 CNNs Lab

Two examples: Look at the following GitHub link:

<https://github.com/yadavrishikesh/Masterclass-V2-2026/blob/main/module2/code> and then the following notebook

1. **CNNs_grayScaleImage.ipynb**: CNNs for digit recognition (example of gray scale image)
2. **CNNs_for_RGBImage_Data.ipynb.ipynb**: CNNs for object detection (Example of of RGB channel)